```
<p>
  <strong>Text:</strong>
  <%= @article.text %>
</p>

<h2>Add a comment:</h2>
<%= form_for([@article, @article.comments.build]) do |f| %>
  <p>
    <%= f.label :commenter %><br>
    <%= f.text_field :commenter %>
  </p>
  <p>
    <%= f.label :body %><br>
    <%= f.text_area :body %>
  </p>
  <p>
    <%= f.submit %>
  </p>
<% end %>

<%= link_to 'Edit', edit_article_path(@article) %> |
<%= link_to 'Back', articles_path %>
```

This adds a form on the `Article` show page that creates a new comment by calling the `CommentsController` `create` action. The `form_for` call here uses an array, which will build a nested route, such as `/articles/1/comments`.

Let's wire up the `create` in `app/controllers/comments_controller.rb`:

```
class CommentsController < ApplicationController
  def create
    @article = Article.find(params[:article_id])
    @comment = @article.comments.create(comment_params)
    redirect_to article_path(@article)
  end

  private
    def comment_params
      params.require(:comment).permit(:commenter, :body)
    end
end
```

You'll see a bit more complexity here than you did in the controller for articles. That's a side-effect of the nesting that you've set up. Each request for a comment has to keep track of the article to which the comment is attached, thus the initial call to the `find` method of the `Article` model to get the article in question.

In addition, the code takes advantage of some of the methods available for an association. We use the `create` method on `@article.comments` to create and save the comment. This will automatically link the comment so that it belongs to that particular article.

Once we have made the new comment, we send the user back to the original article using the `article_path(@article)` helper. As we have already seen, this calls the `show` action of the `ArticlesController` which in turn renders the `show.html.erb` template. This is where we want the comment to show, so let's add that to the `app/views/articles/show.html.erb`.

```
<p>
  <strong>Title:</strong>
  <%= @article.title %>
</p>

<p>
  <strong>Text:</strong>
  <%= @article.text %>
</p>
```